

NumPy Revision Cheatsheet

1) Import + Array Creation

import numpy as np

From list: `a = np.array([1, 2, 3])`

2D array: `A = np.array([[1, 2], [3, 4]])`

Zeros / Ones / Constant:

- `np.zeros((m, n))`
- `np.ones((m, n))`
- `np.full((m, n), k)`

Range:

- `np.arange(start, stop, step)`
- `np.linspace(start, stop, num)`

Identity matrix: `np.eye(n)`

Random:

- `np.random.rand(m, n)` (uniform [0,1))
- `np.random.randn(m, n)` (normal distribution)
- `np.random.randint(low, high, size=(m, n))`

Quick reminder: Prefer NumPy arrays over Python lists for numeric operations (faster + vectorized).

2) Shape, Dimension, and Data Type

`a.shape` ? tuple of dimensions

`a.ndim` ? number of dimensions

`a.size` ? total number of elements

`a.dtype` ? element type

Convert type: `a.astype(np.float64)`

Reshape: `a.reshape(r, c)`

Flatten: `a.ravel()` or `a.flatten()`

Transpose: `A.T` or `A.transpose()`

Formula:

`total_elements = d1 d2 ... dk`

Quick reminder: reshape needs same total elements before/after.

3) Indexing, Slicing, Boolean Masking

Index: $a[i]$, $A[i, j]$

Slice: $a[\text{start}:\text{stop}:\text{step}]$

Row/column:

- $A[i, :]$ (row i)

- $A[:, j]$ (column j)

Boolean filter: $a[a > 0]$

Conditional replacement: $a[a < 0] = 0$

Quick reminder: Slices are usually **views** (not copies). Use `.copy()` when needed.

4) Core Vectorized Operations

For arrays of same shape:

Addition: $C = A + B$

Subtraction: $C = A - B$

Elementwise multiply: $C = A * B$

Elementwise divide: $C = A / B$

Power: A^p

With scalar k :

$A + k$, $A - k$, $A * k$, A / k

Formulae:

Elementwise addition: $C[i,j] = A[i,j] + B[i,j]$

Elementwise multiply: $C[i,j] = A[i,j] * B[i,j]$

Quick reminder: $*$ is elementwise multiplication, not matrix multiplication.

5) Broadcasting (Very Important)

Two shapes are compatible if each trailing dimension is equal or one of them is 1.

Examples:

$(m, n) + (1, n)$?

$(m, n) + (m, 1)$?

$(m, n) + (n,)$?

Formula idea:

If needed, dimension 1 is conceptually repeated to match other shape.

Quick reminder: Read shapes from right to left when checking broadcasting.

6) Aggregations and Statistics

Sum: `np.sum(a) / a.sum()`

Mean: `np.mean(a)`

Median: `np.median(a)`

Standard deviation: `np.std(a)`

Variance: `np.var(a)`

Min/Max: `np.min(a)`, `np.max(a)`

Argmin/Argmax: `np.argmin(a)`, `np.argmax(a)`

Axis usage:

`axis=0` ? column-wise (down rows)

`axis=1` ? row-wise (across columns)

Formulae:

Mean: $\bar{x} = (1/n) \sum x_i$

Variance (population): $\sigma^2 = (1/n) \sum (x_i - \bar{x})^2$

Std dev: $\sigma = \sqrt{\sigma^2}$

Quick reminder: Always confirm axis to avoid wrong output shape.

7) Linear Algebra Essentials

Matrix multiplication: `A @ B` or `np.matmul(A, B)`

Dot product: `np.dot(a, b)`

Determinant: `np.linalg.det(A)`

Inverse: `np.linalg.inv(A)`

Eigenvalues/eigenvectors: `np.linalg.eig(A)`

Solve $Ax=b$: `x = np.linalg.solve(A, b)`

Formulae:

Dot product: $ab = \sum a_i b_i$

Matrix product: $C[i,j] = \sum A[i,k]B[k,j]$

Quick reminder: For solving equations, prefer `solve` over explicit inverse for stability.

8) Stacking, Splitting, and Joining

Vertical stack: `np.vstack([A, B])`

Horizontal stack: `np.hstack([A, B])`

Concatenate: `np.concatenate([A, B], axis=0 or 1)`

Split: `np.split(a, indices_or_sections)`

Quick reminder: Ensure dimensions match except along concatenation axis.

9) Useful Functions for Revision

Unique values: `np.unique(a)`

Sort: `np.sort(a)`

Clip: `np.clip(a, low, high)`

Where condition: `np.where(cond, x, y)`

Any/All:

- `np.any(cond)`
- `np.all(cond)`

Quick reminder: `np.where` is excellent for vectorized if-else logic.

10) Exam-Time Mini Checklist

1. Check shape first: `print(A.shape)`
2. Decide operation type:
 - elementwise (+, -, *, /)
 - matrix (@)
3. Verify broadcasting compatibility
4. Confirm axis in reductions
5. Watch integer vs float dtype
6. Use `.copy()` if modifying slices

11) 30-Second Formula Recap

$\bar{x} = (1/n) \sum x_i$

$\bar{x} = (1/n) \sum (x_i - \bar{x})$

$\bar{x} = \bar{y}$

$ab = \sum a_i b_i$

$C[i,j] = \sum A[i,k]B[k,j]$

Quick reminder note: In NumPy, *think in arrays and shapes*, not loops.